

Unification in Lean4

Maximilian Keßler

April 21, 2026

Index

1. What is unification?
2. Theoretical Background: (simply-typed) λ -calculus
 - 2.1 Decidability of first-order unification
 - 2.2 Undecidability of higher-order unification
3. Unification in Lean
 - 3.1 First-Order unification
 - 3.2 Higher-Order pattern unification

Table of Contents

1. What is unification?

2. Theoretical Background: (simply-typed) λ -calculus

2.1 Decidability of first-order unification

2.2 Undecidability of higher-order unification

3. Unification in Lean

3.1 First-Order unification

3.2 Higher-Order pattern unification

Examples of unification in Lean

```
1 variable (xs ys : List ℕ)
2 -- List.append.{u} {α : Type u} (xs ys : List α) : List α
3 example : List.append xs ys
4 example : @List.append ℕ xs ys
```

Examples of unification in Lean

```
1 variable (xs ys : List ℕ)
2 -- List.append.{u} {α : Type u} (xs ys : List α) : List α
3 example : List.append xs ys
4 example : @List.append ℕ xs ys
```

```
1 example : ∃ (f : ℕ → ℕ), ∀ n, f n = n := ⟨id, fun _ ↦ rfl⟩
2 example : ∃ (f : ℕ → ℕ), ∀ (n : ℕ), f n = n := ⟨id, fun _ ↦ rfl⟩
3 example : ∃ (f : ℕ → ℕ), ∀ n, f n = n := ⟨_, fun _ ↦ rfl⟩
```

Examples of unification in Lean

```
1 variable (xs ys : List ℕ)
2 -- List.append.{u} {α : Type u} (xs ys : List α) : List α
3 example : List.append xs ys
4 example : @List.append ℕ xs ys
```

```
1 example : ∃ (f : ℕ → ℕ), ∀ n, f n = n := ⟨id, fun _ ↦ rfl⟩
2 example : ∃ (f : ℕ → ℕ), ∀ (n : ℕ), f n = n := ⟨id, fun _ ↦ rfl⟩
3 example : ∃ (f : ℕ → ℕ), ∀ n, f n = n := ⟨_, fun _ ↦ rfl⟩
```

```
1 variable (a b : ℕ) (e : a = b) (R : ℕ → ℕ → Prop)
2 -- Eq.subst.{u} {α : Sort u} {motive : α → Prop} {a b : α} (h₁ : a = b) (h₂ : motive
   a) : motive b
3 example (H : R a a)          := Eq.subst e H
4 example (H : R a a) : R a b := @Eq.subst ℕ (R a ·) a b e H
```

Examples of unification in Lean

The `induction` tactic also performs unification:

```
1 variable (n k : ℕ)

3 -- Nat.le_refl (n : ℕ) : n ≤ n
4 -- Nat.le_succ_of_le {n m : ℕ} (h : n ≤ m) : n ≤ m.succ

6 example : n ≤ n + k := by
7   induction k with
8     | zero => exact Nat.le_refl n
9     | succ k ih => exact Nat.le_succ_of_le ih
```

Examples of unification in Lean

The `induction` tactic also performs unification:

```
1 variable (n k : ℕ)

3 -- Nat.le_refl (n : ℕ) : n ≤ n
4 -- Nat.le_succ_of_le {n m : ℕ} (h : n ≤ m) : n ≤ m.succ

6 example : n ≤ n + k := by
7   induction k with
8     | zero => exact Nat.le_refl n
9     | succ k ih => exact Nat.le_succ_of_le ih
```

Lean infers an *induction predicate* (`motive : ℕ → Prop`) to use with the induction principle. The tactic snippet produces the following proof term:

Examples of unification in Lean

The `induction` tactic also performs unification:

```
1 variable (n k : ℕ)

3 -- Nat.le_refl (n : ℕ) : n ≤ n
4 -- Nat.le_succ_of_le {n m : ℕ} (h : n ≤ m) : n ≤ m.succ

6 example : n ≤ n + k := by
7   induction k with
8   | zero => exact Nat.le_refl n
9   | succ k ih => exact Nat.le_succ_of_le ih
```

Lean infers an *induction predicate* (`motive : ℕ → Prop`) to use with the induction principle. The tactic snippet produces the following proof term:

```
1 -- Nat.recAux.{u} {motive : ℕ → Sort u} (zero : motive 0) (succ : (n : ℕ) → motive n
   → motive (n + 1)) (t : ℕ) : motive t
2 example : n ≤ n + k :=
3   @Nat.recAux (n ≤ n + ·) (le_refl n) (fun _ ih ↦ Nat.le_succ_of_le ih) k
```

When does Lean use Unification?

During elaboration, unification infers the types of meta-variables that were inserted while building the syntax tree. For example:

When does Lean use Unification?

During elaboration, unification infers the types of meta-variables that were inserted while building the syntax tree. For example:

- To infer implicit arguments to functions, specified { in brackets }:
 - ▶ Types of arguments of “polymorphic” functions
 - ▶ Motives, typically $\alpha \rightarrow \text{Prop}$

When does Lean use Unification?

During elaboration, unification infers the types of meta-variables that were inserted while building the syntax tree. For example:

- To infer implicit arguments to functions, specified { in brackets }:
 - ▶ Types of arguments of “polymorphic” functions
 - ▶ Motives, typically $\alpha \rightarrow \text{Prop}$
- Types of variables that can be deduced
 - ▶ in bound quantifiers
 - ▶ Type of an object being defined

When does Lean use Unification?

During elaboration, unification infers the types of meta-variables that were inserted while building the syntax tree. For example:

- To infer implicit arguments to functions, specified { in brackets }:
 - ▶ Types of arguments of “polymorphic” functions
 - ▶ Motives, typically $\alpha \rightarrow \text{Prop}$
- Types of variables that can be deduced
 - ▶ in bound quantifiers
 - ▶ Type of an object being defined
- Inferring holes and placeholders

When does Lean use Unification?

During elaboration, unification infers the types of meta-variables that were inserted while building the syntax tree. For example:

- To infer implicit arguments to functions, specified { in brackets }:
 - ▶ Types of arguments of “polymorphic” functions
 - ▶ Motives, typically $\alpha \rightarrow \text{Prop}$
- Types of variables that can be deduced
 - ▶ in bound quantifiers
 - ▶ Type of an object being defined
- Inferring holes and placeholders
- Inferring induction predicates (also a type of motive)
- ...

Table of Contents

1. What is unification?

2. Theoretical Background: (simply-typed) λ -calculus

2.1 Decidability of first-order unification

2.2 Undecidability of higher-order unification

3. Unification in Lean

3.1 First-Order unification

3.2 Higher-Order pattern unification

Simply typed λ -calculus

Definition (Type)

Types are inductively defined as

- Atomic types are types (usually denoted $T, U, V, \dots$)
- When A and B are types, then $A \rightarrow B$ is a type.

Simply typed λ -calculus

Definition (Type)

Types are inductively defined as

- Atomic types are types (usually denoted $T, U, V, \dots$)
- When A and B are types, then $A \rightarrow B$ is a type.

As usual, $A \rightarrow B \rightarrow C := A \rightarrow (B \rightarrow C)$.

Let us assume that we have a (finite) amount of constant symbols, each with a fixed type. Similarly, there is an infinite supply of variable symbols, also with a fixed type.

Simply typed λ -calculus

Definition (Type)

Types are inductively defined as

- Atomic types are types (usually denoted $T, U, V, \dots$)
- When A and B are types, then $A \rightarrow B$ is a type.

As usual, $A \rightarrow B \rightarrow C := A \rightarrow (B \rightarrow C)$.

Let us assume that we have a (finite) amount of constant symbols, each with a fixed type. Similarly, there is an infinite supply of variable symbols, also with a fixed type.

Definition (λ -Term)

λ -*Terms* are inductively defined as

- Constant symbols are terms (usually denoted $a, b, c, \dots$).
- Variables are terms (usually denoted $x, y, z, \dots$).
- If t and u are terms, then $(t \ u)$ is a term.
- If x is a variable and t a term, then $\lambda x \ t$ is a term.

Simply typed λ -calculus

Definition (Type of a Term)

A term t is said to have type T if one of the following holds:

- t is a constant of type T
- t is a variable of type T
- $t = (u \ v)$ and u has type $U \rightarrow T$ and v has type U for some type U .
- $t = \lambda x \ u$, the variable x has type U , the term u has type V and $T = U \rightarrow V$

A term is *well-typed* if it has type T for some T .

Free variables of a term and *substitution* of variables are defined in the usual way. We denote the substitution of the variable x by the term u in the term t by $(u/x)t$.

From now on, let us restrict ourselves to only work with well-typed terms.

Normal forms

Definition (α -conversion)

α -conversion is the act of renaming bound variables (in subterms). That is

$$\lambda x \ t \cong_{\alpha} \lambda y \ (y/x)t$$

if y does not occur in t .

Normal forms

Definition (α -conversion)

α -conversion is the act of renaming bound variables (in subterms). That is

$$\lambda x \ t \cong_{\alpha} \lambda y \ (y/x)t$$

if y does not occur in t .

Definition ($\beta\eta$ -reduction)

$\beta\eta$ -reduction is the recursive (on subterms) and iterative application of the reductions

- β : $(\lambda x \ t)u$ reduces to $(u/x)t$.
- η : $\lambda x \ (t \ x)$ reduces to t if x is not free in t .

Normal forms

Definition (α -conversion)

α -conversion is the act of renaming bound variables (in subterms). That is

$$\lambda x \ t \cong_{\alpha} \lambda y \ (y/x)t$$

if y does not occur in t .

Definition ($\beta\eta$ -reduction)

$\beta\eta$ -reduction is the recursive (on subterms) and iterative application of the reductions

- β : $(\lambda x \ t)u$ reduces to $(u/x)t$.
- η : $\lambda x \ (t \ x)$ reduces to t if x is not free in t .

Theorem

Every well-typed term has a unique $\beta\eta$ -normal form up to α -conversion, called its normal form.

The unification problem

Definition

A *unification problem* is a list of equations, where each equation is a pair of terms t , u . It asks whether there is a substitution θ such that for each equation, the terms θt and θu have the same normal form.

Order of types / unification problems

Definition (Order of a type)

The order of a type is inductively defined as:

- The order of an atomic type T is $o(T) := 1$
- The order of an arrow type $T \rightarrow U$ is $\max(o(T) + 1, o(U))$

Order of types / unification problems

Definition (Order of a type)

The order of a type is inductively defined as:

- The order of an atomic type T is $o(T) := 1$
- The order of an arrow type $T \rightarrow U$ is $\max(o(T) + 1, o(U))$

Example

Let T, U be atomic types.

- $T \rightarrow U$ is of type 2
- $T \rightarrow T \rightarrow U$ is of type 2
- $(T \rightarrow U) \rightarrow T$ is of type 3
- $T \rightarrow (T \rightarrow T) \rightarrow T$ is of type 3

Table of Contents

1. What is unification?

2. Theoretical Background: (simply-typed) λ -calculus

2.1 Decidability of first-order unification

2.2 Undecidability of higher-order unification

3. Unification in Lean

3.1 First-Order unification

3.2 Higher-Order pattern unification

Reduction and Elimination in First-Order unification problems

We present the basic ideas of [Martelli, Montanari, 1982] here, who were the first to give an *efficient* unification algorithm for First-Order unification problems.

Reduction and Elimination in First-Order unification problems

We present the basic ideas of [Martelli, Montanari, 1982] here, who were the first to give an *efficient* unification algorithm for First-Order unification problems.

Theorem

The following operations maintain the solution sets of first-order unification problems:

Reduction and Elimination in First-Order unification problems

We present the basic ideas of [Martelli, Montanari, 1982] here, who were the first to give an *efficient* unification algorithm for First-Order unification problems.

Theorem

The following operations maintain the solution sets of first-order unification problems:

- *Term reduction: Replace an equation*

$$f\ t_1\ t_2\ \cdots\ t_n = f\ u_1\ u_2\ \cdots\ u_n,$$

where f is a function symbol, with the equations $t_1 = u_1, \dots, t_n = u_n$.

Reduction and Elimination in First-Order unification problems

We present the basic ideas of [Martelli, Montanari, 1982] here, who were the first to give an *efficient* unification algorithm for First-Order unification problems.

Theorem

The following operations maintain the solution sets of first-order unification problems:

- *Term reduction: Replace an equation*

$$f\ t_1\ t_2\ \cdots\ t_n = f\ u_1\ u_2\ \cdots\ u_n,$$

where f is a function symbol, with the equations $t_1 = u_1, \dots, t_n = u_n$.

- *Variable elimination: If $x = t$ is an equation, where x is a variable and t any term, then apply the substitution $\{(t, x)\}$ to both terms of all other equations.*

A unification algorithm

Algorithm

Given a set of equations, repeatedly perform any of the following transformations. If no transformation applies, stop with success.

- *Select an equation $t = x$, where t is not a variable and x is a variable, and rewrite it to $x = t$.*

A unification algorithm

Algorithm

Given a set of equations, repeatedly perform any of the following transformations. If no transformation applies, stop with success.

- *Select an equation $t = x$, where t is not a variable and x is a variable, and rewrite it to $x = t$.*
- *Erasing an equation $x = x$, where x is a variable.*

A unification algorithm

Algorithm

Given a set of equations, repeatedly perform any of the following transformations. If no transformation applies, stop with success.

- *Select an equation $t = x$, where t is not a variable and x is a variable, and rewrite it to $x = t$.*
- *Erasing an equation $x = x$, where x is a variable.*
- *Select an equation $t = u$, where t and u are not variables. If the root function symbols differ, stop with failure. Otherwise, apply term reduction.*

A unification algorithm

Algorithm

Given a set of equations, repeatedly perform any of the following transformations. If no transformation applies, stop with success.

- *Select an equation $t = x$, where t is not a variable and x is a variable, and rewrite it to $x = t$.*
- *Erasing an equation $x = x$, where x is a variable.*
- *Select an equation $t = u$, where t and u are not variables. If the root function symbols differ, stop with failure. Otherwise, apply term reduction.*
- *Select an equation of the form $x = t$, where x is a variable (which occurs somewhere else in the equation set) and where $t \neq x$. If x occurs in t , stop with failure. Otherwise, apply variable elimination.*

A unification algorithm

Algorithm

Given a set of equations, repeatedly perform any of the following transformations. If no transformation applies, stop with success.

- *Select an equation $t = x$, where t is not a variable and x is a variable, and rewrite it to $x = t$.*
- *Erasing an equation $x = x$, where x is a variable.*
- *Select an equation $t = u$, where t and u are not variables. If the root function symbols differ, stop with failure. Otherwise, apply term reduction.*
- *Select an equation of the form $x = t$, where x is a variable (which occurs somewhere else in the equation set) and where $t \neq x$. If x occurs in t , stop with failure. Otherwise, apply variable elimination.*

Theorem (Martelli, Montanari, 1982)

The above algorithm terminates and represents a non-deterministic decision procedure for First-Order unification.

Table of Contents

1. What is unification?

2. Theoretical Background: (simply-typed) λ -calculus

2.1 Decidability of first-order unification

2.2 Undecidability of higher-order unification

3. Unification in Lean

3.1 First-Order unification

3.2 Higher-Order pattern unification

Hilbert's tenth problem

Theorem

The (higher-order) unification problem in simply typed λ -calculus is undecidable.

Theorem (Matiyacevich-Robinson-Davis [Matiyacevich 1970, Davis 1973])

Hilbert's tenth problem is undecidable, i.e. there is no algorithm taking as input two Polynomials $P(X_1, \dots, X_n)$, $Q(X_1, \dots, X_n)$ with natural coefficients and decides whether there are natural numbers $m_1, \dots, m_n$ such that

$$P(m_1, \dots, m_n) = Q(m_1, \dots, m_n).$$

Church natural numbers

Definition

For a natural number n , denote by $\bar{n}$ its *Church number*

$$\bar{n} := \lambda x \lambda f \underbrace{(f(f(\dots(f \ x) \dots)))}_{n \text{ times}}$$

Note that this has type $T \rightarrow (T \rightarrow T) \rightarrow T$.

Church natural numbers

Definition

For a natural number n , denote by $\bar{n}$ its *Church number*

$$\bar{n} := \lambda x \lambda f \underbrace{(f(f(\dots(f \ x) \dots)))}_{n \text{ times}}$$

Note that this has type $T \rightarrow (T \rightarrow T) \rightarrow T$. Also define the terms

$$+ := \lambda n \lambda m \lambda x \lambda f \ (n \ (m \times f) \ f)$$

$$\cdot := \lambda n \lambda m \lambda x \lambda f \ (n \times (\lambda z \ m \ z \ f))$$

of type $(T \rightarrow (T \rightarrow T) \rightarrow T) \rightarrow (T \rightarrow (T \rightarrow T) \rightarrow T) \rightarrow T \rightarrow (T \rightarrow T) \rightarrow T$.

Church natural numbers

Definition

For a natural number n , denote by $\bar{n}$ its *Church number*

$$\bar{n} := \lambda x \lambda f \underbrace{(f(f(\dots(f \ x) \dots)))}_{n \text{ times}}$$

Note that this has type $T \rightarrow (T \rightarrow T) \rightarrow T$. Also define the terms

$$\begin{aligned} + &:= \lambda n \lambda m \lambda x \lambda f \ (n \ (m \ x \ f) \ f) \\ \cdot &:= \lambda n \lambda m \lambda x \lambda f \ (n \ x \ (\lambda z \ m \ z \ f)) \end{aligned}$$

of type $(T \rightarrow (T \rightarrow T) \rightarrow T) \rightarrow (T \rightarrow (T \rightarrow T) \rightarrow T) \rightarrow T \rightarrow (T \rightarrow T) \rightarrow T$.

Theorem (Addition, Multiplication)

Let $n, m \in \mathbb{N}$. The normal form of $\bar{n} + \bar{m}$ is $\overline{n + m}$. The normal form of $\bar{n} \cdot \bar{m}$ is $\overline{n \cdot m}$. In particular, for every polynomial $P \in \mathbb{N}[X_1, \dots, X_n]$, there is a term p such that the normal form of $(p \ \bar{m}_1 \ \dots \ \bar{m}_n)$ is $\overline{P(m_1, \dots, m_n)}$.

Undecidability of Higher-Order unification

Theorem

The Higher-Order unification problem (in simply-typed λ -calculus) is undecidable.

Proof (Sketch).

By reduction from Hilbert's tenth problem.

Undecidability of Higher-Order unification

Theorem

The Higher-Order unification problem (in simply-typed λ -calculus) is undecidable.

Proof (Sketch).

By reduction from Hilbert's tenth problem. Let $P, Q \in \mathbb{N}[X_1, \dots, X_n]$ be polynomials, and let p, q be their corresponding λ -terms. Then, consider the unification problem

$$(p \ X_1 \ \cdots \ X_n) = (q \ X_1 \ \cdots \ X_n).$$

If P and Q have a (natural) point in common, then this has a solution.

Undecidability of Higher-Order unification

Theorem

The Higher-Order unification problem (in simply-typed λ -calculus) is undecidable.

Proof (Sketch).

By reduction from Hilbert's tenth problem. Let $P, Q \in \mathbb{N}[X_1, \dots, X_n]$ be polynomials, and let p, q be their corresponding λ -terms. Then, consider the unification problem

$$(p \ X_1 \ \cdots \ X_n) = (q \ X_1 \ \cdots \ X_n).$$

If P and Q have a (natural) point in common, then this has a solution. By adding additional unification constraints that force the X_i to be Church numbers, we can make the converse true. $\square$

Table of Contents

1. What is unification?

2. Theoretical Background: (simply-typed) λ -calculus

2.1 Decidability of first-order unification

2.2 Undecidability of higher-order unification

3. Unification in Lean

3.1 First-Order unification

3.2 Higher-Order pattern unification

How to approach unification in Lean

We cannot hope to have a general algorithm for unification. Practical approach :

How to approach unification in Lean

We cannot hope to have a general algorithm for unification. Practical approach :

- Implement First-Order unification
- Use Higher-Order *pattern unification* (using Miller-patterns) to solve a decidable subset of unification constraints
- Do not tackle the rest of problems, i.e. elaboration fails in other cases

Table of Contents

1. What is unification?

2. Theoretical Background: (simply-typed) λ -calculus

2.1 Decidability of first-order unification

2.2 Undecidability of higher-order unification

3. Unification in Lean

3.1 First-Order unification

3.2 Higher-Order pattern unification

First-Order unification in Lean

Lean's elaborator implements First-Order unification. However, compared to the λ -calculus, it also has to take into account:

First-Order unification in Lean

Lean's elaborator implements First-Order unification. However, compared to the λ -calculus, it also has to take into account:

- Dependent types and type universes
- Handling definitional equality (instead of $\beta\eta$ -normal form)
- Inserting *coercions* to “convert” between types
- Respecting transparency levels regarding unfolding behavior of definitions
- Typeclass synthesis

First-Order unification in Lean

Lean's elaborator implements First-Order unification. However, compared to the λ -calculus, it also has to take into account:

- Dependent types and type universes
- Handling definitional equality (instead of $\beta\eta$ -normal form)
- Inserting *coercions* to “convert” between types
- Respecting transparency levels regarding unfolding behavior of definitions
- Typeclass synthesis

Fortunately, this is still decidable. Lean4 has a quite complex algorithm to handle all of the above.

Examples of first-order unification

Let us walk through an example in more detail.

```
1 variable (a b c : ℕ)
2 -- Eq.trans.{u} {α : Sort u} {a b c : α} (h₁ : a = b) (h₂ : b = c) : a = c
3 example (h₁ : a = b) (h₂ : b = c) := Eq.trans h₁ h₂ -- Has type a = c
```

Examples of first-order unification

Let us walk through an example in more detail.

```
1 variable (a b c : ℕ)
2 -- Eq.trans.{u} {α : Sort u} {a b c : α} (h₁ : a = b) (h₂ : b = c) : a = c
3 example (h₁ : a = b) (h₂ : b = c) := Eq.trans h₁ h₂ -- Has type a = c
```

First, Lean instantiates metavariables ($?α : \text{Sort } u$) ($?a ?b ?c : ?α$) and creates the expression.

```
1 @Eq.trans ?α ?a ?b ?c h₁ h₂ : ?a = ?c
```

Examples of first-order unification

Let us walk through an example in more detail.

```
1 variable (a b c : ℕ)
2 -- Eq.trans.{u} {α : Sort u} {a b c : α} (h₁ : a = b) (h₂ : b = c) : a = c
3 example (h₁ : a = b) (h₂ : b = c) := Eq.trans h₁ h₂ -- Has type a = c
```

First, Lean instantiates metavariables $(?α : \text{Sort } u)$ $(?a ?b ?c : ?α)$ and creates the expression.

```
1 @Eq.trans ?α ?a ?b ?c h₁ h₂ : ?a = ?c
```

For this to be well-typed, we get the constraints

```
1 h₁ : Eq a b = Eq ?a ?b
2 h₂ : Eq b c = Eq ?b ?c
```

Examples of first-order unification

Let us walk through an example in more detail.

```
1 variable (a b c : ℕ)
2 -- Eq.trans.{u} {α : Sort u} {a b c : α} (h₁ : a = b) (h₂ : b = c) : a = c
3 example (h₁ : a = b) (h₂ : b = c) := Eq.trans h₁ h₂ -- Has type a = c
```

First, Lean instantiates metavariables $(?α : \text{Sort } u)$ $(?a ?b ?c : ?α)$ and creates the expression.

```
1 @Eq.trans ?α ?a ?b ?c h₁ h₂ : ?a = ?c
```

For this to be well-typed, we get the constraints

```
1 h₁ : Eq a b = Eq ?a ?b
2 h₂ : Eq b c = Eq ?b ?c
```

which will be reduced to deduce $?a = a$, $?b = b$, $?c = c$.

Examples of first-order unification

Let us walk through an example in more detail.

```
1 variable (a b c : ℕ)
2 -- Eq.trans.{u} {α : Sort u} {a b c : α} (h₁ : a = b) (h₂ : b = c) : a = c
3 example (h₁ : a = b) (h₂ : b = c) := Eq.trans h₁ h₂ -- Has type a = c
```

First, Lean instantiates metavariables $(?α : \text{Sort } u)$ $(?a ?b ?c : ?α)$ and creates the expression.

```
1 @Eq.trans ?α ?a ?b ?c h₁ h₂ : ?a = ?c
```

For this to be well-typed, we get the constraints

```
1 h₁ : Eq a b = Eq ?a ?b
2 h₂ : Eq b c = Eq ?b ?c
```

which will be reduced to deduce $?a = a$, $?b = b$, $?c = c$. Then, we also get $?α = \mathbb{N}$, since $?a = a : \mathbb{N}$.

Examples of first-order unification

Let us walk through an example in more detail.

```
1 variable (a b c : ℕ)
2 -- Eq.trans.{u} {α : Sort u} {a b c : α} (h₁ : a = b) (h₂ : b = c) : a = c
3 example (h₁ : a = b) (h₂ : b = c) := Eq.trans h₁ h₂ -- Has type a = c
```

First, Lean instantiates metavariables $(?α : \text{Sort } u)$ $(?a ?b ?c : ?α)$ and creates the expression.

```
1 @Eq.trans ?α ?a ?b ?c h₁ h₂ : ?a = ?c
```

For this to be well-typed, we get the constraints

```
1 h₁ : Eq a b = Eq ?a ?b
2 h₂ : Eq b c = Eq ?b ?c
```

which will be reduced to deduce $?a = a$, $?b = b$, $?c = c$. Then, we also get $?α = \mathbb{N}$, since $?a = a : \mathbb{N}$. Finally, we deduce that the type of the expression is $\text{Eq } ?a ?c = \text{Eq } a c$. Note that Lean still checks that b and c are also of type $α$.

Table of Contents

1. What is unification?
2. Theoretical Background: (simply-typed) λ -calculus
 - 2.1 Decidability of first-order unification
 - 2.2 Undecidability of higher-order unification
3. Unification in Lean
 - 3.1 First-Order unification
 - 3.2 Higher-Order pattern unification

Patterns

Definition (Miller 1991)

A *pattern* is a higher-order unification problem where all meta-variables occurring in a term are applied to some distinct and bound variables. In such a case, we can use a λ -expression for the metavariable.

Example

Let us denote metavariables by X , Y . The following is an example of a pattern:

- $\lambda x \lambda y \lambda z (X \ x \ y) = \lambda x \lambda y \lambda z (x \ (\text{suc } y))$

with solution $X = \lambda x \lambda y (x \ (\text{suc } y))$.

Example

The following are *not* examples of patterns:

- $\lambda x \lambda y \lambda z (X \ x \ x \ y) = \lambda x \lambda y \lambda z (x \ (\text{suc } y))$ (non-linear)
- $\lambda x \lambda y \lambda z (X \ (Y \ x) \ y) = \lambda x \lambda y \lambda z (x \ (\text{suc } y))$ (applied to other meta-variable)
- $\lambda x \lambda y \lambda z (X \ x \ (\text{suc } y)) = \lambda x \lambda y \lambda z (x \ (\text{suc } y))$ (applied to non-variable term)

Note that in all cases, the unification problem is still solvable (but not using patterns).

Examples of higher-order pattern unification

```
1 example:  $\exists F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}), \forall g\ x, (F\ g)\ x = g\ (g\ x) := \langle \_, \text{fun } \_ \_ => \text{rfl} \rangle$   
2 -- rfl.{u} {α : Sort u} {a : α} : a = a
```

Examples of higher-order pattern unification

```
1 example:  $\exists F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}), \forall g\ x, (F\ g)\ x = g\ (g\ x) := \langle \_, \text{fun } \_ \_ => \text{rfl} \rangle$   
2 -- rfl.{u} { $\alpha$  : Sort u} {a :  $\alpha$ } : a = a
```

We will get metavariables $(?F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}))$, $(?\alpha : \text{Sort } u)$ and $(?a : ?\alpha)$. Naming the holes on the right by $(g : \mathbb{N} \rightarrow \mathbb{N})$, $(x : \mathbb{N})$, we get the following unification problem:

```
1 Eq (?F g x) (g (g x)) = Eq ?a ?a
```

Examples of higher-order pattern unification

```
1 example:  $\exists F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}), \forall g\ x, (F\ g)\ x = g\ (g\ x) := \langle \_, \text{fun } \_ \_ => \text{rfl} \rangle$   
2 -- rfl.{u} { $\alpha$  : Sort u} {a :  $\alpha$ } : a = a
```

We will get metavariables $(?F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}))$, $(?\alpha : \text{Sort } u)$ and $(?a : ?\alpha)$. Naming the holes on the right by $(g : \mathbb{N} \rightarrow \mathbb{N})$, $(x : \mathbb{N})$, we get the following unification problem:

```
1 Eq (?F g x) (g (g x)) = Eq ?a ?a
```

Lean will reduce this to $?a = ?F\ g\ x$ and $?a = g\ (g\ x)$, thus

```
1 ?F g x = g (g x)
```

which is a Miller-pattern.

Examples of higher-order pattern unification

```
1 example:  $\exists F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}), \forall g\ x, (F\ g)\ x = g\ (g\ x) := \langle \_, \text{fun } \_ \_ => \text{rfl} \rangle$   
2 -- rfl.{u} { $\alpha$  : Sort u} {a :  $\alpha$ } : a = a
```

We will get metavariables $(?F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}))$, $(?\alpha : \text{Sort } u)$ and $(?a : ?\alpha)$. Naming the holes on the right by $(g : \mathbb{N} \rightarrow \mathbb{N})$, $(x : \mathbb{N})$, we get the following unification problem:

```
1 Eq (?F g x) (g (g x)) = Eq ?a ?a
```

Lean will reduce this to $?a = ?F\ g\ x$ and $?a = g\ (g\ x)$, thus

```
1 ?F g x = g (g x)
```

which is a Miller-pattern. So by pattern-matching Lean can infer

```
1 ?F := fun g x  $\mapsto$  g (g x)
```

which solves the constraint.

Examples of higher-order pattern unification

```
1 example:  $\exists F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}), \forall g\ x, (F\ g)\ x = g\ (g\ x) := \langle \_, \text{fun } \_ \_ => \text{rfl} \rangle$   
2 -- rfl.{u} { $\alpha$  : Sort u} {a :  $\alpha$ } : a = a
```

We will get metavariables $(?F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N}))$, $(?\alpha : \text{Sort } u)$ and $(?a : ?\alpha)$. Naming the holes on the right by $(g : \mathbb{N} \rightarrow \mathbb{N})$, $(x : \mathbb{N})$, we get the following unification problem:

```
1 Eq (?F g x) (g (g x)) = Eq ?a ?a
```

Lean will reduce this to $?a = ?F\ g\ x$ and $?a = g\ (g\ x)$, thus

```
1 ?F g x = g (g x)
```

which is a Miller-pattern. So by pattern-matching Lean can infer

```
1 ?F := fun g x  $\mapsto$  g (g x)
```

which solves the constraint. $a = ?F\ g\ x$ and $?\alpha = \mathbb{N}$ follow from First-Order unification.

Thank you for your attention.

Feel free to ask any more questions.